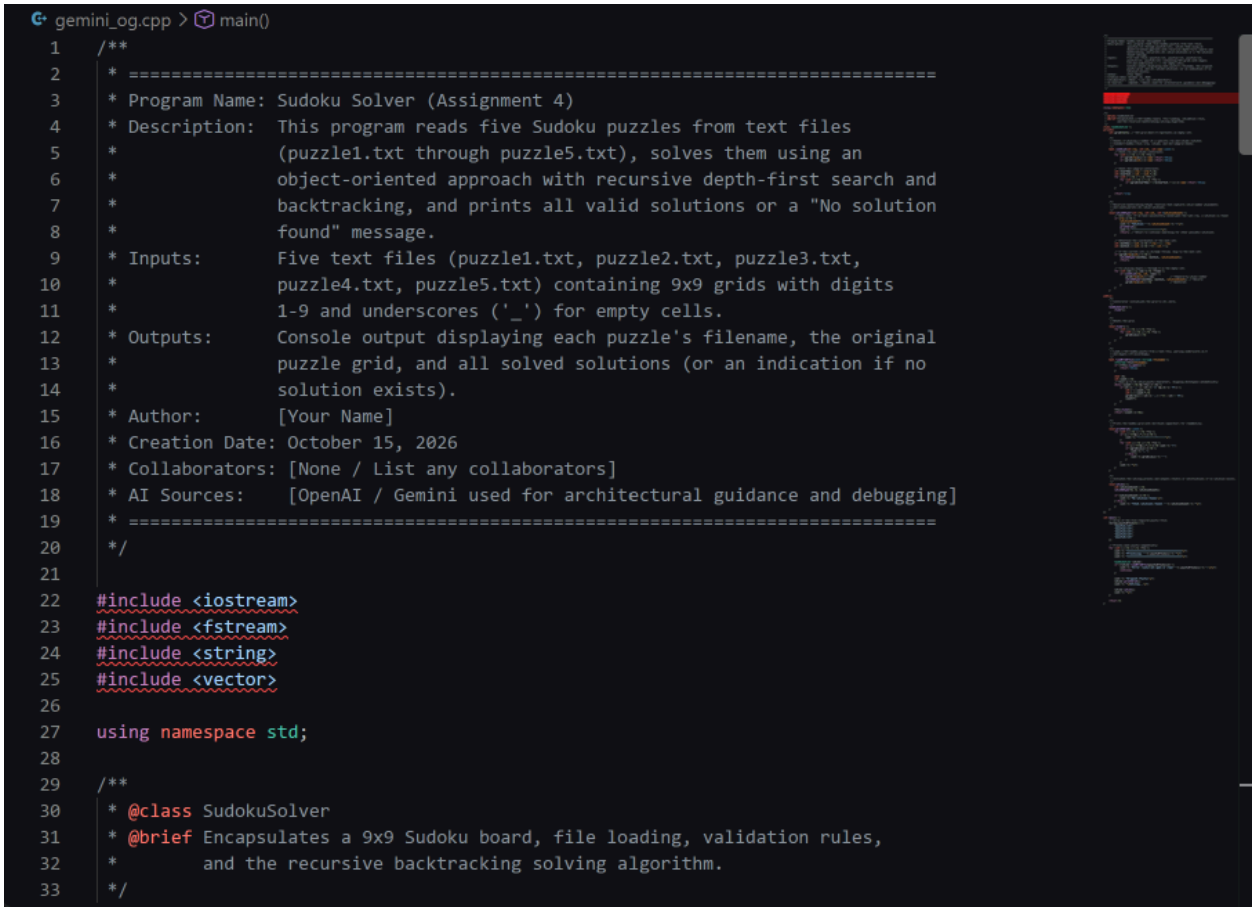
Parker Curl 3105711

Gemini Original Code:



```
gemini_og.cpp > main()
/**
 * ============================================================================
 * Program Name: Sudoku Solver (Assignment 4)
 * Description:  This program reads five Sudoku puzzles from text files
 *               (puzzle1.txt through puzzle5.txt), solves them using an
 *               object-oriented approach with recursive depth-first search and
 *               backtracking, and prints all valid solutions or a "No solution
 *               found" message.
 * Inputs:       Five text files (puzzle1.txt, puzzle2.txt, puzzle3.txt,
 *               puzzle4.txt, puzzle5.txt) containing 9x9 grids with digits
 *               1-9 and underscores ('_') for empty cells.
 * Outputs:      Console output displaying each puzzle's filename, the original
 *               puzzle grid, and all solved solutions (or an indication if no
 *               solution exists).
 * Author:       [Your Name]
 * Creation Date: October 15, 2026
 * Collaborators: [None / List any collaborators]
 * AI Sources:    [OpenAI / Gemini used for architectural guidance and debugging]
 * ============================================================================
 */

#include <iostream>
#include <fstream>
#include <string>
#include <vector>

using namespace std;

/**
 * @class SudokuSolver
 * @brief Encapsulates a 9x9 Sudoku board, file loading, validation rules,
 *        and the recursive backtracking solving algorithm.
 */
```

```cpp
class SudokuSolver {
private:
    int grid[9][9]; // 9x9 grid where 0 represents an empty cell

    /**
     * Checks if placing a number at a specific row and column violates
     * standard Sudoku rules (row, column, and 3x3 subgrid check).
     */
    bool isValid(int row, int col, int num) const {
        // Check row and column constraints
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == num) return false;
            if (grid[i][col] == num) return false;
        }

        // Check 3x3 subgrid constraints
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (grid[startRow + r][startCol + c] == num) return false;
            }
        }

        return true;
    }

    /**
     * Recursive backtracking helper function that explores valid number placements
     * and counts/prints all valid solutions.
     */
    void solveHelper(int row, int col, int &solutionCount) {
```

```
66          // Base case: If we have successfully moved past the last row, a solution is found
67          if (row == 9) {
68              solutionCount++;
69              cout << "Solution " << solutionCount << ":\n";
70              printGrid();
71              cout << "---------------------\n";
72              return; // Return to continue searching for other possible solutions
73          }
74
75          // Determine the coordinates of the next cell
76          int nextRow = (col == 8) ? row + 1 : row;
77          int nextCol = (col == 8) ? 0 : col + 1;
78
79          // If the current cell is already filled, skip to the next cell
80          if (grid[row][col] != 0) {
81              solveHelper(nextRow, nextCol, solutionCount);
82              return;
83          }
84
85          // Try placing digits 1 through 9 in the empty cell
86          for (int num = 1; num <= 9; ++num) {
87              if (isValid(row, col, num)) {
88                  grid[row][col] = num;           // Temporarily place number
89                  solveHelper(nextRow, nextCol, solutionCount); // Recurse
90                  grid[row][col] = 0;             // Backtrack
91              }
92          }
93      }
94
95  public:
```

```
96      /**
97       * Constructor initializes the grid to all zeros.
98       */
99      SudokuSolver() {
100         clear();
101     }
102
103     /**
104      * Resets the grid.
105      */
106     void clear() {
107         for (int i = 0; i < 9; ++i) {
108             for (int j = 0; j < 9; ++j) {
109                 grid[i][j] = 0;
110             }
111         }
112     }
113
114     /**
115      * Loads a 9x9 Sudoku puzzle from a text file, parsing underscores as 0
116      * and digits 1-9 accordingly.
117      */
118     bool loadFromFile(const string& filename) {
119         ifstream file(filename);
120         if (!file.is_open()) {
121             return false;
122         }
123
124         char ch;
125         int count = 0;
```

```
126         // Read up to 81 valid puzzle characters, skipping whitespace automatically
127         while (count < 81 && file >> ch) {
128             if (ch == '_' || (ch >= '1' && ch <= '9')) {
129                 int r = count / 9;
130                 int c = count % 9;
131                 grid[r][c] = (ch == '_') ? 0 : (ch - '0');
132                 count++;
133             }
134         }
135 
136         file.close();
137         return (count == 81);
138     }
139 
140     /**
141      * Prints the Sudoku grid with 3x3 block separators for readability.
142      */
143     void printGrid() const {
144         for (int i = 0; i < 9; ++i) {
145             if (i > 0 && i % 3 == 0) {
146                 cout << "------+------+------\n";
147             }
148             for (int j = 0; j < 9; ++j) {
149                 if (j > 0 && j % 3 == 0) cout << "|";
150                 if (grid[i][j] == 0) {
151                     cout << "_ ";
152                 } else {
153                     cout << grid[i][j] << " ";
154                 }
155             }
```

```
            cout << "\n";
        }
    }

    /**
     * Initiates the solving process and outputs results or notifications if no solution exist
     */
    void solve() {
        int solutionCount = 0;
        solveHelper(0, 0, solutionCount);

        if (solutionCount == 0) {
            cout << "No solution found.\n";
        } else {
            cout << "Total solutions found: " << solutionCount << "\n";
        }
    }
};

int main() {
    // Array of the five required puzzle files
    string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Process each puzzle sequentially
    for (int i = 0; i < 5; ++i) {
        cout << "========================================\n";
        cout << "Processing: " << puzzleFiles[i] << "\n";
        cout << "========================================\n";

        SudokuSolver solver;
        if (!solver.loadFromFile(puzzleFiles[i])) {
            cout << "Error: Could not open or read " << puzzleFiles[i] << ".\n\n";
            continue;
        }

        cout << "Original Puzzle:\n";
        solver.printGrid();
        cout << "\nSolving...\n";

        solver.solve();
        cout << "\n";
    }

    return 0;
}
```

Access: Gemini Via Web

Prompt: Write a complete C++ program for EECS 348 Software Engineering I Assignment 4 that solves Sudoku puzzles using object-oriented programming and recursive backtracking.

The program must meet the following requirements:

- Use C++ classes and objects to organize the program, rather than relying on standalone functions.
- Read five Sudoku puzzle files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
- Each puzzle contains a 9x9 grid with digits 1–9 and underscores (_) representing empty cells.
- Follow standard Sudoku rules: every row, column, and 3x3 block must contain digits 1–9 exactly once.
- Use recursive depth-first search with backtracking to solve each puzzle by trying valid numbers in empty cells.
- Print the puzzle filename and original puzzle before solving it.
- Print the completed solution or "No solution found" if the puzzle cannot be solved.
- If a puzzle has multiple valid solutions, print every solution.
- Make sure the program can compile and run using g++ on the University of Kansas EECS Cycle Linux servers.
- Put all C++ code in a single file named Assignment4.cpp.

Include a prologue comment containing the program name, description, inputs, outputs, author, creation date, collaborators, and AI sources. Include comments explaining the major sections of the program and how the code works.

Keep the code understandable for a college student taking an introductory Software Engineering course.

Provide the complete C++ source code and a brief explanation of how your solution works.

Claude Original Code:

claude_og.cpp > ...

```cpp
/*
 * ============================================================================
 * Program Name  : Assignment4.cpp
 * Description   : Solves Sudoku puzzles using object-oriented programming and
 *                 recursive depth-first search with backtracking. The program
 *                 finds EVERY valid solution for a puzzle (not just the first),
 *                 and reports "No solution found" when none exist.
 * Inputs        : Five text files in the current directory:
 *                 puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 *                 Each file holds a 9x9 grid. Digits 1-9 are given numbers and
 *                 an underscore (_) is an empty cell. Spaces/newlines between
 *                 characters are ignored, so "5 3 _ ..." and "53_..." both work.
 * Outputs       : For each puzzle (printed to the console):
 *                   - the puzzle's filename
 *                   - the original puzzle
 *                   - every solution found, or "No solution found"
 * Author        : Parker Curl
 * Creation Date : October 15, 2026
 * Collaborators : None
 * AI Sources    : Claude (Anthropic) was used to help draft this program
 *                 structure, the backtracking logic, and the comments.
 *
 * Compile : g++ -std=c++11 -Wall -o Assignment4 Assignment4.cpp
 * Run     : ./Assignment4
 * ============================================================================
 */

#include <iostream>
#include <fstream>
#include <string>
#include <vector>

using namespace std;
```

```
34
35  // ------------------------------------------------------------------------
36  // Class: SudokuPuzzle
37  // Represents one 9x9 Sudoku board. It knows how to load itself from a file,
38  // print itself, and check whether placing a number in a cell follows the rules.
39  // ------------------------------------------------------------------------
40  class SudokuPuzzle {
41  public:
42      static const int SIZE = 9;        // The board is 9 x 9
43      static const int BLOCK = 3;       // Each sub-box is 3 x 3
44      static const int EMPTY = 0;       // We store empty cells as 0 internally
45
46      // Start with every cell empty
47      SudokuPuzzle() {
48          for (int r = 0; r < SIZE; r++)
49              for (int c = 0; c < SIZE; c++)
50                  grid[r][c] = EMPTY;
51      }
52
53      // Read the puzzle from a file. Returns false if the file can't be opened
54      // or doesn't contain exactly 81 valid cells.
55      bool loadFromFile(const string& filename) {
56          ifstream in(filename.c_str());
57          if (!in) {
58              return false;
59          }
60
61          int count = 0;       // How many cells we've read so far
62          char ch;
63          while (in.get(ch)) {
64              if (ch >= '1' && ch <= '9') {
65                  if (count >= SIZE * SIZE) return false;   // too many cells
66                  grid[count / SIZE][count % SIZE] = ch - '0';
```

```
67                  count++;
68              } else if (ch == '_') {
69                  if (count >= SIZE * SIZE) return false;   // too many cells
70                  grid[count / SIZE][count % SIZE] = EMPTY;
71                  count++;
72              }
73              // Anything else (spaces, newlines, tabs, '\r') is skipped.
74          }
75          return count == SIZE * SIZE;
76      }
77
78      // Print the board, showing '_' for empty cells
79      void print() const {
80          for (int r = 0; r < SIZE; r++) {
81              for (int c = 0; c < SIZE; c++) {
82                  if (grid[r][c] == EMPTY) cout << '_';
83                  else cout << grid[r][c];
84                  if (c < SIZE - 1) cout << ' ';
85              }
86              cout << endl;
87          }
88      }
89
90      // Get / set a single cell
91      int get(int row, int col) const { return grid[row][col]; }
92      void set(int row, int col, int value) { grid[row][col] = value; }
```

```
93
94      // Can 'value' be placed at (row, col) without breaking a Sudoku rule?
95      // We check the row, the column, and the 3x3 block.
96      // (The cell itself is skipped so this also works on already-filled cells.)
97      bool isValidPlacement(int row, int col, int value) const {
98          // Check the row and the column
99          for (int i = 0; i < SIZE; i++) {
100             if (i != col && grid[row][i] == value) return false;
101             if (i != row && grid[i][col] == value) return false;
102         }
103
104         // Check the 3x3 block: find the block's top-left corner first
105         int startRow = (row / BLOCK) * BLOCK;
106         int startCol = (col / BLOCK) * BLOCK;
107         for (int r = startRow; r < startRow + BLOCK; r++) {
108             for (int c = startCol; c < startCol + BLOCK; c++) {
109                 if ((r != row || c != col) && grid[r][c] == value) return false;
110             }
111         }
112         return true;
113     }
114
115     // Do the numbers already given in the puzzle follow the rules?
116     // If two givens conflict, the puzzle can never be solved.
117     bool givensAreValid() const {
118         for (int r = 0; r < SIZE; r++) {
119             for (int c = 0; c < SIZE; c++) {
120                 if (grid[r][c] != EMPTY &&
121                     !isValidPlacement(r, c, grid[r][c])) {
122                     return false;
123                 }
```

```
124             }
125         }
126         return true;
127     }
128
129     // Find the first empty cell (scanning left-to-right, top-to-bottom).
130     // Returns true and fills row/col if one exists; false if the board is full.
131     bool findEmptyCell(int& row, int& col) const {
132         for (int r = 0; r < SIZE; r++) {
133             for (int c = 0; c < SIZE; c++) {
134                 if (grid[r][c] == EMPTY) {
135                     row = r;
136                     col = c;
137                     return true;
138                 }
139             }
140         }
141         return false;
142     }
143
144 private:
145     int grid[SIZE][SIZE];   // 0 = empty, 1-9 = a number
146 };
147
148 // ----------------------------------------------------------------------------
149 // Class: SudokuSolver
150 // Solves a SudokuPuzzle using recursive depth-first search with backtracking
151 // and prints every solution it finds.
```

```cpp
// ---------------------------------------------------------------------------
class SudokuSolver {
public:
    // Solve the given puzzle and print all solutions.
    // We work on a copy so the caller's original puzzle isn't changed.
    void solveAndPrintAll(const SudokuPuzzle& original) {
        solutionCount = 0;
        SudokuPuzzle working = original;

        // If the starting numbers already break the rules, skip the search.
        if (working.givensAreValid()) {
            search(working);
        }

        if (solutionCount == 0) {
            cout << "No solution found" << endl;
        } else if (solutionCount > 1) {
            cout << "Total solutions found: " << solutionCount << endl;
        }
    }

private:
    int solutionCount;   // How many solutions we've found for this puzzle

    // The recursive heart of the program.
    //
    // How it works:
    //   1. Find an empty cell. If there are none, the board is complete,
    //      so we've found a solution: print it.
    //   2. Otherwise try the numbers 1-9 in that cell. For each number that
    //      is legal, place it and recursively try to solve the rest.
    //   3. After the recursive call returns (whether or not it found
```

```
184     //      anything), REMOVE the number (backtrack) and try the next one.
185     //      Because we keep going instead of stopping at the first solution,
186     //      we discover every solution.
187     void search(SudokuPuzzle& puzzle) {
188         int row, col;
189
190         // Base case: no empty cells left -> the puzzle is solved
191         if (!puzzle.findEmptyCell(row, col)) {
192             solutionCount++;
193             cout << "Solution " << solutionCount << ":" << endl;
194             puzzle.print();
195             cout << endl;
196             return;
197         }
198
199         // Recursive case: try each digit in the empty cell
200         for (int value = 1; value <= 9; value++) {
201             if (puzzle.isValidPlacement(row, col, value)) {
202                 puzzle.set(row, col, value);          // make a choice
203                 search(puzzle);                        // explore deeper
204                 puzzle.set(row, col, SudokuPuzzle::EMPTY);  // undo (backtrack)
205             }
206         }
207         // If no digit worked, we simply return, which sends the previous
208         // call back to try its next digit.
209     }
210 };
211
212 // ------------------------------------------------------------------------
213 // Class: SudokuApp
```

```
213  // Class: SudokuApp
214  // Controls the overall program: loops over the five puzzle files, loads each
215  // one, prints it, and asks the solver to solve it.
216  // ------------------------------------------------------------------------
217  class SudokuApp {
218  public:
219      SudokuApp() {
220          // The five input files required by the assignment
221          files.push_back("puzzle1.txt");
222          files.push_back("puzzle2.txt");
223          files.push_back("puzzle3.txt");
224          files.push_back("puzzle4.txt");
225          files.push_back("puzzle5.txt");
226      }
227
228      void run() {
229          for (size_t i = 0; i < files.size(); i++) {
230              processPuzzle(files[i]);
231          }
232      }
233
234  private:
235      vector<string> files;   // Names of the puzzle files
236      SudokuSolver solver;    // Does the actual solving
237
238      // Handle one puzzle file from start to finish
239      void processPuzzle(const string& filename) {
240          cout << "============================" << endl;
241          cout << "File: " << filename << endl;
242          cout << "============================" << endl;
243
244          SudokuPuzzle puzzle;
```

```
245         if (!puzzle.loadFromFile(filename)) {
246             cout << "Error: could not read a valid 9x9 puzzle from "
247                  << filename << endl << endl;
248             return;
249         }
250
251         cout << "Original puzzle:" << endl;
252         puzzle.print();
253         cout << endl;
254
255         solver.solveAndPrintAll(puzzle);
256         cout << endl;
257     }
258 };
259
260 // ----------------------------------------------------------------------------
261 // main: create the application object and run it.
262 // ----------------------------------------------------------------------------
263 int main() {
264     SudokuApp app;
265     app.run();
266     return 0;
267 }
```

Access: Claude via Web

Prompt: Write a complete C++ program for EECS 348 Software Engineering I Assignment 4 that solves Sudoku puzzles using object-oriented programming and recursive backtracking.

The program must meet the following requirements:

- Use C++ classes and objects to organize the program, rather than relying on standalone functions.
- Read five Sudoku puzzle files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt.
- Each puzzle contains a 9x9 grid with digits 1–9 and underscores (_) representing empty cells.
- Follow standard Sudoku rules: every row, column, and 3x3 block must contain digits 1–9 exactly once.
- Use recursive depth-first search with backtracking to solve each puzzle by trying valid numbers in empty cells.
- Print the puzzle filename and original puzzle before solving it.
- Print the completed solution or "No solution found" if the puzzle cannot be solved.
- If a puzzle has multiple valid solutions, print every solution.
- Make sure the program can compile and run using g++ on the University of Kansas EECS Cycle Linux servers.
- Put all C++ code in a single file named Assignment4.cpp.

Include a prologue comment containing the program name, description, inputs, outputs, author, creation date, collaborators, and AI sources. Include comments explaining the major sections of the program and how the code works.

Keep the code understandable for a college student taking an introductory Software Engineering course.

Provide the complete C++ source code and a brief explanation of how your solution works.

Comparison & Analysis

Correctness: Both Claude and Gemini created programs that solve sudoku puzzles using recursive backtracking. Both also print all possible solutions instead of just stopping at the first one. The biggest difference I noticed was that Claude checks the starting numbers to make sure they don't already break sodokus rules. Gemini doesn't have this check, Claude also made sure that there are exactly 81 cells in the puzzle, while gemini stops reading after 81 cells even if there is extra data. Because of this, I think Claude handles incorrect puzzle inputs better.

Execution Time: The Gen AI's both use recursive backtracking, so they may have to try several different numbers before finding a solution. Claude searches for the first empty cell each time, while Gemini moves through the puzzle in order. Neither program originally had a way to choose which empty square would be easiest to solve first. For harder puzzles, both programs could take longer because they may have to try a lot of different number combinations before finding the right one. I would expect their execution times to be fairly similar, but i havent measured the original programs to confirm that.

Space Complexity: They both use a 9x9 array to store the sudoku board, so the board itself uses a fixed amount of memory. They also use extra memory when recursive calls are made. Claude makes a copy of the original puzzle before solving, while Gemini changes the board directly and then backtracks. Claude uses slightly more memory because of the copy, but the difference is small.

Maintainability: Claude's program was easier for me to follow because it separated everything into three classes, (sodkoku puzzle, sudokuSolver, and SodokuApp) each handles a different part of the program. Gemini used one class for most of the work, with main() handling the files. Gemini's code was shorter, but I think claudes organization makes it easier to find specific sections and make changes.

Program Selection & Improvements

Program Selection: I chose Claude's program as the base for my final version. Both programs used backtracking to solve the puzzles, but Claude had better input checking and separated the code into three classes. I thought this made it easier to understand and improve Gemini's version.

Improvements made: I made a few changes to Claude's original program with the help from ChatGPT. First, I improved the file checking so the program would reject unexpected characters instead of just skipping them. I also made sure it wouldn't keep partially loaded puzzle data if something went wrong while reading the file. The biggest change was how the program picks an empty square. Claude originally just chose the first empty square it found. I changed this to use MRV, which picks the empty square with the fewest possible numbers. This helps the program avoid trying as many unnecessary combinations. The last change was adding a check that stops searching a path if an empty square has no possible numbers. Some comments were also added.

Effect of Improvements: The file checking changes improved correctness because the program now catches more problems with the puzzle files. The MRV change can improve execution time because the program spends less time trying numbers that won't work. It doesn't guarantee every puzzle will be faster, but it should help on more difficult puzzles. The space complexity stayed about the same since the program still uses 9x9 array and recursive backtracking. The changes only added a small amount of extra memory. For maintainability, I kept claude’s 3 class structure and made the comments easier to follow. This helps us know what the program is actually doing, but also makes it easier to find things if we want to make changes later.